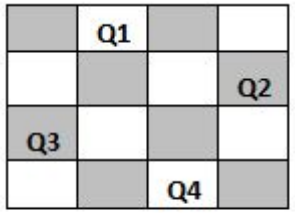
AI Lab
Week

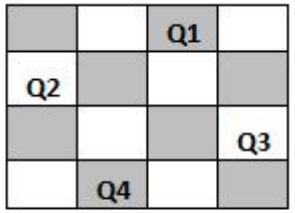
Implement Hill Climbing search algorithm to solve N-Queens problem

The 4-Queens Problem

- 4 - Queens problem is to place 4-queens in such a manner on an 4x4 chessboard that no queens attack each other by being in the same row, column or diagonal.



Solution 1



Solution 2

Hill-climbing search algorithm

```
function HILL-CLIMBING(problem) returns a state that is a local maximum

    current ← MAKE-NODE(problem.INITIAL-STATE)
    loop do
        neighbor ← a highest-valued successor of current
        if neighbor.VALUE ≤ current.VALUE then return current.STATE
        current ← neighbor
```

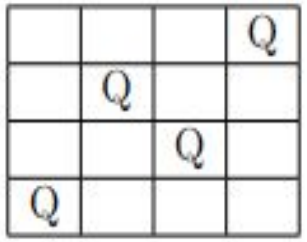
- State: 4 queens on the board. One queen per column.
  - Variables: x_0, x_1, x_2, x_3 where x_i is the row position of the queen in column i. Assume that there is one queen per column.
  - Domain for each variable: $x_i \in \{0, 1, 2, 3\}, \forall i$.
- Initial state: a random state.
- Goal state: 4 queens on the board. No pair of queens are attacking each other.
- Neighbour relation:

  Swap the row positions of two queens.
- Cost function: The number of pairs of queens attacking each other, directly or indirectly.

Observation Book

- In the observation, show the cost calculation of current state and neighbor nodes. And continue until you reach goal configuration of 4-queens board.

Start with the initial state $x_0 = 3, x_1 = 1, x_2 = 2, x_3 = 0$. Show the steps of executing the hill climbing algorithm until it terminates.



If multiple neighbours have the same cost, choose the neighbour where the pair of queens swapped has the smallest subscript/column number. For example, when we can swap either (x_0, x_4) or (x_2, x_3), we will swap (x_0, x_4). When we can swap either (x_2, x_3) or (x_2, x_4) we will swap (x_2, x_3).